

Ajanik Kurumsal Araştırma Sistemleri: Kanıt Temelli Bir Tasarım Çerçevesi

Ömer Faruk Yavuz

August 2026

Özet

Bu çalışma, yazılım mühendisliği organizasyonlarında kurumsal bilginin sohbet odaklı araçlarda erimesi problemini tanımlar ve buna karşı çok ajanlı bir araştırma sistemi önerir. Önerilen çerçevenin ayırt edici iddiası, çok ajanlı mimarilerin temel kazanımının işlem hızı değil, kanıtların birbirinden bağımsız kaynaklardan toplanarak çapraz doğrulanması olduğudur. Çalışma; bilgi katmanlarının ayrıştırılması, kanıt güven sıralaması, paralelleştirme ölçütü, iddia (claim) şeması, çelişki çözüm kuralı, doğrulama mekanizmaları ve sistemin uzun vadeli sürdürülebilirlik koşullarını kapsar.

Problem Tanımı

Bilgi Modelinin Ontolojik Konumu

Kurumsal bilgi sistemlerinin değerlendirilmesinde belirleyici ölçüt, bilginin bir *akış* olarak mı yoksa bir *varlık* olarak mı ele alındığıdır. Akış modelinde bilgi zaman içinde ilerleyen bir mesaj dizisidir ve geçmişe doğru erişim, dizinin geriye taranmasıyla mümkündür. Varlık modelinde ise bilgi, kendi kimliği, adresi ve ilişkileri olan kalıcı bir nesnedir.

Varlık modelinin gerektirdiği nitelikler şu zincirde toplanır:

Bilgi $\rightarrow$ *Adreslenebilir* $\rightarrow$ *Aranabilir* $\rightarrow$ *Versiyonlu* $\rightarrow$ *Bağlantılı* $\rightarrow$ *Kalıcı*

Bu zincirin işletilebilirlik ölçütü tek bir soruya indirgenebilir: *Altı ay sonra bu bilgi tek bir aramayla bulunabilir mi?* Olumsuz yanıt, ilgili sistemin bilgi tutma kapasitesinin zayıf olduğunu gösterir.

Karşılaştırmalı Gözlem

Tablo yapıları araçlarda her verinin bir adresi bulunur; satır, sütun, hücre, tablo ve sayfa hiyerarşisi sayesinde belirli bir hücre referansı tüm kullanıcılar için aynı nesneye işaret eder. Bilgi bu modelde konuşulmaz, modellenir. Elektronik posta sistemleri ise sohbet uygulaması değil iletişim veritabanıdır: gönderen,

alıcı, tarih, konu, thread, ek, arama ve etiket alanları sayesinde kayıtlar yıllar sonra dahi sorgulanabilir ve thread hiyerarsisi bağlamı korur.

Buna karşılık *conversation-first* tasarımlarda bilgi ikinci plandadır. Aynı konuya ilişkin veri özel mesajlar, kanal mesajları, toplantı sohbetleri, emoji ile verilmiş onaylar ve harici dosya paylaşımları arasında parçalanır. Mesaj akışı merkezde olduğu için, birkaç hafta sonra zincirdeki ara halkanın içeriği geri getirilemez hale gelir.

Devir Teslim Problemi

Problemin en keskin biçimde görünür olduğu durum, ekibe yeni katılan bir mühendisin devam eden işi devralmasıdır. Yeni mühendisin görevi, bir yıllık mesaj geçmişini geriye kaydırarak okumak olamaz. Bunun üç gerekçesi vardır: içeriğin büyük bölümü ilgisizdir, ilgili mesajın bulunması güvenilir bir arama stratejisiyle desteklenmez ve kararların gerekçesi çoğunlukla hiçbir yerde yazılı değildir.

Kötü işleyen senaryoda soru zinciri şu biçimde tıkanır: konunun geçmişte konuşulduğu hatırlanır, bağlantı bulunmaz, kimin yazdığı bilinmez. Bu, kurumsal hafızanın kaybıdır. İyi işleyen senaryoda ise her soru kalıcı bir artefakta çözülür: karar gerekçesi bir mimari karar kaydına, arayüz tasarımı bir tasarım dokümanına, dağıtım süreci bir runbook'a, hata geçmişi bir issue kaydına, kodun biçimi bir PR tartışmasına.

Bilgi Katmanları

Katman	Barındığı ortam	Kalıcılık
Anlık koordinasyon	Sohbet araçları	Yok
Kararlar	ADR, Wiki	Var
Gereksinimler	Issue tracker, dokümanlar	Var
Kodun gerekçesi	Kod, yorum, tasarım dokümanı	Var
Değişiklik geçmişi	Commit, PR	Var
İşletim yordamı	Runbook, playbook	Var

Sohbet araçları koordinasyon işlevi görür; bilginin barınağı değildir. Havacılık, uzay, savunma ve finans sektörlerindeki büyük mühendislik organizasyonlarında sohbet araçları bulunmakla birlikte, değerli bilgi dokümantasyon, sürüm kontrolü, karar kayıtları, test sonuçları ve izlenebilir kayıtlarda tutulur.

Araştırma Protokolü

Bir araştırma ajanının kaynaklara erişim sırası rastgele değil, kanıt güvenilirliği ölçütüne göre belirlenir.

Adımlar

1. Problemin tipini belirle: hata, özellik, mimari karar, iş kuralı veya operasyonel süreç.

2. Ontolojiyi çıkar: ilgili varlıklar ve aralarındaki ilişkiler.
3. Kodu tara: metin araması, sözdizim ağacı analizi, sembol araması, arayüz ve tip tanımları.
4. Sürüm geçmişini incele: `git blame`, `git log`, `git show`.
5. Değişiklik taleplerini oku: tasarım gerekçeleri çoğunlukla yalnızca burada yazılıdır.
6. Gereksinim kayıtlarına bak: ilk talep ve kabul kriterleri.
7. Dokümantasyonu tara: README, Wiki, ADR, RFC, tasarım dokümanı, runbook.
8. Testleri oku: sistemin beklenen davranışını bildiren en kesin kaynaktır.
9. Çalıştır: log, hata ayıklama, kesme noktası, iz sürme.
10. İnsana danış: en son adım; insan en pahalı kaynaktır.
11. Kanıt güven puanını hesapla.
12. Çelişki analizi yürüt.
13. Bilgi grafiğini oluştur.

Onuncu adımın en sona konulması bir maliyet tercihidir; ilk dokuz adımın tükenmesi, insana yöneltilecek sorunun dar ve cevaplanabilir olmasını sağlar.

Bilgi Grafiği

Her araştırma sonunda problem düğümlüne bağlı bir graf oluşur: gereksinim kaydı, değişiklik talebi, commit, test, sınıf, fonksiyon, arayüz, yapılandırma, veritabanı şeması, doküman, log ve sorumlu kişi. Bu noktadan sonra yapılan iş arama değil keşiftir.

Kanıt Güven Modeli

Sayısal Ölçek

Güven, kaynak tipinden gelen taban değer ile üç düzeltme çarpanının bileşimi olarak tanımlanır:

$$C = B \cdot f_t \cdot f_v \cdot f_c, \quad C \in [0, 1]$$

Burada B kaynak tipi taban değeri, f_t tazelik çarpanı, f_v doğrulama çarpanı, f_c tutarlılık çarpanıdır.

Kaynak tipi	Taban değer B
Çalışan kod	0.95
Test	0.95
Üretim logu	0.90
Değişiklik talebi tartışması	0.75
Mimari karar kaydı	0.75
Gereksinim kaydı	0.65
Wiki	0.50
Sohbet kaydı	0.30
Beyana dayalı hatırlama	0.15
Onaylanmamış ajan raporu	0.10

Tazelik çarpanı, kaynağın geçersizleşme hızına bağlı yarı ömür üzerinden hesaplanır:

$$f_t = 2^{-\Delta t/h}$$

Δt kaynağın son değişikliğinden bu yana geçen süre, h ise kaynak tipine özgü yarı ömürdür. Üretim logu için saatler, sürüm geçmişi için depo başındaki değişiklik olayı, doküman için haftalar, gereksinim kaydı için aylar mertebesinde belirlenir.

Doğrulama çarpanı f_v , iddianın konum referansının programatik olarak doğrulanıp doğrulanmadığına göre 1.0 veya 0.0 değerini alır. Doğrulanamayan iddia sisteme girmez.

Tutarlılık çarpanı f_c , aynı varlığa ilişkin bağımsız kaynaklardan gelen destek sayısına göre artar, çelişki varlığında azalır:

$$f_c = \min(1.0, 0.7 + 0.15 \cdot n_{destek}) \cdot (1 - 0.25 \cdot n_{celiski})$$

Bu formülasyonun amacı sayısal kesinlik iddiası taşımak değil, kanıt değerlendirmesini ajanın serbest yargısından çıkarıp denetlenebilir bir hesaba bağlamaktır.

Paralleleştirme Mimarisi

Ayrıştırma Ölçütü

Bir adım, ancak üç koşulu birlikte sağlıyorsa paralel yürütülebilir: girdisi başka bir ajanın çıktısına bağlı olmamalı, yan etkisi bulunmamalı ve çıktısı sentezden bağımsız olarak anlamlı olmalıdır.

SIRALI

```

Problem tipini belirle      <- her seyin girdisi
Ontolojiyi cikar            <- arama terimlerini uretir
...
Celiski analizi             <- tum bulgular gerekli
Guyen puanlama              <- tum bulgular gerekli
Sonuc raporu

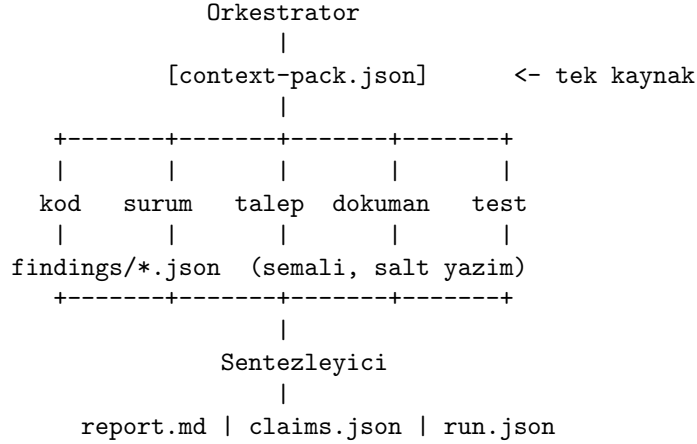
```

PARALEL

Kod | Surum | Talep | Gereksinim | Dokuman | Test | Log

Ontoloji adımının paralelleştirilmesi sistemi bozar: ortak sözlük kurulmadan çalışan ajanlar aynı varlığı farklı terimlerle arar ve bulguları birleştirilemez.

Bileşen Yapısı



Ortak Bağlam Paketi

Fan-out öncesinde üretilen ve her ajana değiştirilmeksizin verilen pakettir.

```
{
  "question": "...",
  "kind": "architectural-decision",
  "entities": ["Download", "Cut", "Library", "S3"],
  "aliases": { "Download": ["downloadAudio", "download_count"] },
  "time_window": "2025-06-01..2026-08-01",
  "repos": ["repo-a", "repo-b"],
  "out_of_scope": ["..."]
}
```

`aliases` alanı ajanlar arası terim hizalamasını sağlar ve paketin en kritik bileşenidir.

Uzmanlaşma

Ajanlar tek uzmanlık ilkesiyle tanımlanır: kod ajanı ilgili dosya, fonksiyon ve çağrı zincirini çıkarır; sürüm ajanı commit geçmişi ve sorumluluk atamalarını inceler; talep ajanı tasarım gerekçelerini okur; gereksinim ajanı kabul kriterlerini toplar; doküman ajanı wiki, karar kaydı ve teknik önerileri tarar; test ajanı

testlerin garanti ettiği davranışı özetler; log ajanı gerçek çalışma davranışını gözler. Tümü tek bir sentez ajanına rapor verir.

Rol kısıtının gevşetilmesi ajanları aynı kaynağa yöneltir; bağımsızlık kaybolduğunda çapraz doğrulama anlamını yitirir. Paralelliğin değeri hızda değil kaynak bağımsızlığındadır.

Başlangıç konfigürasyonu olarak üç ajan yeterlidir: sistemin ne yaptığını bildiren kod ajanı, neden böyle olduğunu bildiren geçmiş ajanı ve ne yapması beklendiğini bildiren bilgi ajanı. Aranan cevap çoğunlukla birinci ile üçüncünün ayrıştığı noktada bulunur.

Soru Tipine Göre Ajan Seti

Sabit bir yürütme grafiği bütün sorulara uymaz. Problem tipi alanı, koşacak ajan kümesini belirler.

Problem tipi	Baskın kaynaklar
Hata	Log, test, sürüm geçmişi
Mimari karar	Değişiklik talebi, karar kaydı, gereksinim
İş kuralı	Gereksinim, doküman, test
Operasyonel	Runbook, yapılandırma, log
Devir teslim	Derinlik yerine kapsam genişliği

Eşleme, komut metni içinde değil yapılandırma dosyasında tutulur.

Paralelliğin Uygun Olmadığı Durumlar

Soru dar kapsamlı olduğunda, ontoloji belirsiz kaldığında, kaynakların çoğu erişilemez durumdayken veya cevabın konumu zaten biliniyorken fan-out zarar üretir. N ajan N katı bağlam maliyeti anlamına gelir ve kazanç yalnızca kaynaklar gerçekten bağımsızsa doğar.

İddia Şeması ve Doğrulama

Biçimsel Tanım

Ajanlar birbiriyle doğrudan haberleşmez; paylaşılan depolama üzerinden sabit şemayla iletişim kurar. Şema hem sentezin girdisi hem araştırma kaydının kendisidir.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["agent", "run_id", "claims", "gaps", "sources_checked"],
  "properties": {
    "agent": { "enum": ["code", "history", "knowledge", "people",
                        "execution", "evidence", "contradiction"] },
    "run_id": { "type": "string" },
  },
}
```

```

"claims": {
  "type": "array",
  "items": {
    "type": "object",
    "required": ["id", "statement", "evidence", "as_of",
      "confidence", "entities", "sensitivity"],
    "properties": {
      "id": { "type": "string" },
      "statement": { "type": "string", "maxLength": 400 },
      "evidence": {
        "type": "object",
        "required": ["type", "ref", "locator"],
        "properties": {
          "type": { "enum": ["code", "test", "log", "pr",
            "adr", "issue", "wiki"] },
          "ref": { "type": "string" },
          "locator": { "type": "string",
            "pattern": "^[^:]+:L[0-9]+(-L[0-9]+)?$" }
        }
      },
      "as_of": { "type": "string", "format": "date" },
      "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
      "entities": { "type": "array", "items": { "type": "string" } },
      "sensitivity": { "enum": ["public", "internal", "restricted"] }
    }
  },
  "candidate_entities": { "type": "array", "items": { "type": "string" } },
  "gaps": { "type": "array", "items": { "type": "string" } },
  "dead_ends": { "type": "array", "items": { "type": "string" } },
  "rejected": { "type": "array", "items": { "type": "string" } },
  "sources_checked": { "type": "array", "items": { "type": "string" } },
  "sources_useless": { "type": "array", "items": { "type": "string" } }
}

```

Konum Doğrulayıcı

Konum referansı alanı zorunludur ve şema düzeyinde biçimsel olarak sınırlandırılmıştır. Doğrulayıcı, sentez öncesinde her iddia için şu işlemi yürütür: referans edilen dosya ve satır aralığı açılır, içerik okunur, iddia metniyle anlamsal örtüşme sınanır. Örtüşme sağlanamayan iddia rapora girmez ve **gaps** listesine gerekçesiyle taşınır.

Doğrulayıcının çıktısı aynı zamanda sistemin temel kalite göstergesidir. Doğrulama oranı

$$R_{loc} = \frac{\text{doğrulanan iddiasayısı}}{\text{üretilen iddiasayısı}}$$

halüsinasyon oranının doğrudan ve programatik olarak ölçülebilen tersidir. Bu oranın eşik altına düşmesi, ilgili ajanın komut tanımının gözden geçirilmesini gerektirir.

Çelişki Çözümü

Tespit

Çelişki tespiti dil modeline bırakılmaz; deterministik eşleştirmeyeyle yürütülür. Aynı varlığa dokunan iddialar gruplanır, aynı zaman kesitine ait olanlar karşılaştırılır, uyumsuzluk halinde kayıt açılır.

Zaman Ekseninin Ayırıştırılması

Farklı kaynaklar farklı zaman kesitlerine aittir. Dokümanla kod arasındaki fark her durumda çelişki değildir; doküman yazıldığı anda doğru olup sonradan geçersizleşmiş olabilir. `as_of` alanı bu ayrımı taşır. İki iddianın zaman damgaları belirgin biçimde ayrışıyorsa sonuç çelişki değil *evrim* olarak raporlanır. Araştırmanın aradığı cevap çoğunlukla bu evrim kaydının kendisidir.

Karar Kuralı

Çelişki kaydı açıldığında şu sıra işletilir:

1. Zaman kesitleri ayrışıyorsa kayıt evrim olarak sınıflandırılır ve kapatılır.
2. Kesitler aynıysa iddialar güven puanına göre sıralanır.
3. Puan farkı 0.30 eşiğini aşıyorsa yüksek puanlı iddia *birincil*, diğeri *itiraz* olarak etiketlenir; kayıt açık kalır.
4. Puan farkı eşik altındaysa iddialar eşdeğer sayılır ve kayıt insan hakemliğine yönlendirilir.
5. Hiçbir durumda kayıt otomatik kapatılmaz.

Bu kuralın gerekçesi şudur: yüksek güvenli kaynak, doğru olan kaynak değil, doğrulanması kolay olan kaynaktır. Otomatik kapatmaya izin veren bir sistem, hatalı çalışan kodu doğru yazılmış gereksinime tercih eder ve hatayı kurumsal hafızaya gerçek olarak kaydeder.

Dayanıklılık Önlemleri

Önbellek ve Geçersizleştirme

Bulgular bağlam paketinin özet değerine bağlanır. Kaynakların geçersizleşme hızı türe göre farklılaştığından tek bir yaşam süresi yerine kaynak başına geçersizleştirme kuralı tanımlanır: sürüm bulguları depo başı değiştiğinde, doküman bulguları ilgili sayfa güncellendiğinde, log bulguları saat mertebesinde geçersizleşir.

Bütçe ve Erken Durdurma

Her ajana azami araç çağırısı, azami bağlam tüketimi, süre sınırı ve doygunluk koşulu tanımlanır. Bütçesini tüketip iddia üretemeyen ajan başarısız sayılmaz; ürettiği yokluk bildirimi rapora yazılır. Sessizce düşen ajan, var olmayan bilginin var sanılmasına yol açar.

Kısmi Başarısızlık Politikası

Üç politikadan biri önceden seçilir: tümünün başarısını şart koşan katı yaklaşım, eldeki bulguyla devam eden ve eksikleri işaretleyen varsayılan yaklaşım, kritik ajanları zorunlu tutup kalanı isteğe bağlı bırakan yeter sayı yaklaşımı. Raporun başında kapsam bildirimi yer alır: taranan kaynak sayısı ve eksiklerin gerekçesi. Bu bildirim olmadan okuyucu raporu tam sanar.

Eşzamanlı Araştırmalar ve Hız Sınırı

Birden fazla araştırma aynı anda yürüdüğünde kaynak rekabeti, önbellek çakışması ve çelişkili rapor sorunları doğar. Dış servis kotaları ajan başına değil sistem genelinde yönetilir. Uygulanan model, kaynak başına tek bir jeton kovaasıdır: kova kapasitesi servis kotasının altında tutulur, doldurma hızı sabittir ve tüketim önceliği araştırmanın yaş sırasına göre belirlenir; böylece geç başlayan bir araştırma erken başlayana aç bırakmaz. Kova boşaldığında ajan hata vermez, bekler; bekleme süresi bütçeyi aşarsa ilgili kaynak **gaps** kaydına kota gerekçesiyle yazılır.

Varlık bazlı bir izin tutulur; yeni bir raporun iddiası mevcut bir iddiayla çelişiyorsa her ikisi de işaretlenir. Aksi halde kurumsal hafızaya birbiriyle çelişen iki gerçek girer.

Tekrar Üretilbilirlik

Rapor tek başına yetersizdir. Koşu kaydı ayrıca saklanır: koşu kimliği, bağlam paketi özeti, depo başı, ajan durumları, model sürümü ve örnekleme parametreleri. Bu kayıt olmadan ajanik araştırma, eleştirdiği sohbet aracının konumuna düşer: bir yerde üretilmiş, gerekçesi kaybolmuş bilgi.

Doğruluk ve Güvenlik

Yetki Sınırı

Ajanlar orkestratörün kimliğiyle değil, çağırın kullanıcının yetkisiyle koşar. Erişilemeyen kaynak sessizce atlanmaz, yetki gerekçesiyle kaydedilir. İddialar kaynak hassasiyetine göre etiketlenir ve rapor görünümüleri bu etikete göre süzülür. Kural özellikle müşteri verisinin bulunduğu ortamlarda kritiktir: güven sıralamasında en üstte yer alan üretim logu, aynı zamanda en hassas veri kümesidir.

Enjeksiyon Yüzeyi

Ajanın okuduğu her içerik — gereksinim açıklaması, değişiklik talebi yorumu, wiki sayfası, commit mesajı, kod yorumu — dışarıdan yazılabilir metindir ve talimat değil veri olarak işlenir. İşçi ajanların araç erişimi salt okumayla sınırlandırılır. Sentez ajanı ham metni değil şemalı iddiaları görür; böylece şemalı ara katman güvenlik işlevi de üstlenir. Talimat niteliği taşıyan metin iddia olarak değil, şüphe kaydı olarak raporlanır.

Varlık Kimliği

Ajan yeni bir varlık keşfettiğinde bağlam paketini değiştiremez; aday varlık olarak raporlar. Sentez ajanı adayları mevcut ontolojiye bağlar veya reddeder; kabul edilenler bir sonraki koşunun eşanlamlı tablosuna girer. Ontoloji koşular arasında büyür, koşu içinde sabit kalır.

Kendi Çıktısını Kaynak Sayma

Uzun vadedeki en sinsî başarısızlık biçimi budur. İkinci koşu birinci koşunun raporunu kanıt sayarsa, birkaç yinleme sonunda dayanağı olmayan bir iddia bağımsız kaynaklarca doğrulanmış görünümü kazanır. Önlem iki katmanlıdır: onaylanmamış ajan raporunun taban güven değeri en alt seviyede tutulur ve işçi ajanların tarama alanından rapor dizini çıkarılır. Yalnızca sentez ajanı, yalnızca yinleme kontrolü amacıyla önceki raporlara erişir.

Sorunun Öncülü

Protokol, sorulan sorunun doğru soru olduğunu varsayar. Uygulamada en sık karşılaşılan kayıp, doğru cevaplanmış yanlış sorudur. Sentez ajanının son denetimi, toplanan bulguların sorunun öncülüünü doğrulayıp doğrulamadığıdır. Öncül bulgularla çelişiyorsa cevap üretilmeden önce soru yeniden formüle edilir. Öncülü sessizce kabul eden sistem kendini tutarlı biçimde yanıltır.

Kurumsal Hafızaya Geri Besleme

Negatif Bilginin Kaydı

Eksiklik kaydı tek başına yetersizdir; kaydedilmesi gereken üç ayrı kategori vardır. Eksiklik, aranan fakat bulunamayan bilgiyi gösterir. Çıkmaz yol, de-

nenmiş ve çürütölmüş hipotezi kaydeder. Reddedilmiş çözüm ise geçmişte değerdendirilip terk edilmiş yaklaşımı belgeler. İkinci ve üçüncü kategoriler kaydedilmezse her yeni araştırma aynı çıkmaza yeniden girer. Bu, insan ekiplerinde de en sistem-atik biçimde kaybedilen bilgi türüdür.

Öneri Üretimi

Sentez adımı kalıcı kaynaklara doğrudan yazma yetkisine sahip değildir; öneri üretir. Eksiklikler karar kaydı taslağına, çelişkiler doküman düzeltme talebine, işlevsiz kaynaklar kullanımdan kaldırma önerisine dönüşür. Otomatik birleştirme yapılmaz; ajanın ürettiğı bilgi kendi güven tablosunda en alt sıradadır ve insan onayı almadan yukarı taşınmaz.

İnsana Danışma Adımının Tasarımı

Soru tek bir kişiye yöneltilir; sorumlu kişi, kod sahipliğı kayıtlarından belirlenir. Soru cevaplanabilir biçimde daraltılır ve hangi kaynakların tarandığı, neyin bulunamadığı bağlamıyla birlikte sunulur. Alınan cevap doğrudan kalıcı kaynağına yazılır, sohbet ortamına değil. Aksi halde en pahalı kaynaktan elde edilen bilgi geçici bir ortamda ölür ve sistemin çözme-yi amaçladığı döngüye geri dönölür.

Rapor Görünümleri

Çıktının en az üç farklı tüketicisi vardır: anlık okuyan insan, aylar sonra okuyan insan ve sonraki koşudaki ajan. Birincisi kısa, kanıt bağlantılı ve çelişkileri öne alan bir metin ister; ikincisi bağlamı ve o günkü depo durumunu içeren bir kayıt; üçüncüsü yapılandırılmış ve güven puanlı iddia listesi. Üçü tek dosyada birleştirilirse her biri bozulur. Aynı veriden üç görünüm üretilir: insan raporu, iddia listesi ve koşu kaydı.

Ölçüm

Sistemin işlevselliğı dört gösterge üzerinden izlenir: konum doğrulama oranı, insana danışma oranı, sonradan insan tarafından tespit edilen çelişki sayısı ve işaretlenen eksiğın kapanma süresi. Birincisi tümüyle otomatiktir ve doğrudan kalite göstergesidir. Sonuncusu ise sistemin kurumsal öğrenmeye dönüşüp dönüşmediğini gösterir; eksikler işaretlenip kapanmıyorsa sistem yalnızca borç biriktiriyordur.

Sürdürülebilirlik ve Sahiplik

Tasarımın en sessiz açığı teknik değildir. Ontoloji güncellenmediğinde, eşanlamlı tablosu bakımsız kaldığında ve problem tipi eşlemesi eskidiğinde sistem birkaç ay içinde gürültü üretmeye başlar; güvenilirliğini yitirdiğı anda da terk edilir.

Açıkça sahiplenilmesi gereken sorumluluklar şunlardır: ontolojinin onayı ve güncellenmesi, aday varlıkların kabul veya reddi, önerilen karar kayıtlarının kalıcı kaynağına birleştirilmesi, konum doğrulama oranı eşik altına düştüğünde müdahale, hız sınırı politikasının servis kotalarıyla uyumunun izlenmesi.

Sahipsiz bir bilgi sistemi, kaybolan bilginin daha pahalı bir versiyonudur. Sohbet odaklı araçların problemi de esasen teknik değil sahiplik problemidir; burada önerilen sistem, aynı nedenden ötürü aynı biçimde yürüyebilir.

Sonuç

Bu çalışma, kurumsal bilginin sohbet akışlarında değil kalıcı ve adreslenebilir artefaktlarda yaşaması gerektiği tezinden hareketle çok ajanlı bir araştırma sisteminin tasarımını ortaya koymuştur. Tasarım dört katmanda yığılır: temel yapı sistemi kurar, dayanıklılık önlemleri işletim koşullarında çökmesini engeller, doğruluk önlemleri yanlış sonuç üretmesini sınırlar, sürdürülebilirlik önlemleri zaman içinde terk edilmesini geciktirir.

Çalışmanın merkezi savı şudur: ajanik sistemlerde asıl kazanım paralellik miktarının artırılması değil, kanıtların birbirinden bağımsız kaynaklardan toplanarak karşılıklı olarak doğrulanması veya yürütülmesidir. Konum doğrulaması, zaman ekseninin ayrıştırılması ve çelişkilerin otomatik kapatılmaması bu savın işletebilir karşılıklarıdır. Sistemin nihai kırılabilirliği ise algoritmik değil kurumsaldır: sahipliği tanımlanmadığı takdirde geri kalan bütün önlemler işlevsizleşir.